

TOP Queries

Select upper(first_name) as student_name from student;

Select distinct major from student

Select major from student group by (major);

Select substring(first_name,1,3) from Student;

Select instr(lower(first_name),'a') from student where first_name = 'Shivansh';

Select distinct major, length(major) as length from student;

Select Replace (first_name, 'a', 'A') from student;

Select concat(first_name+' '+last_name) as Complete_name from student;

Select * from student order by first_name, major desc;

Select * from student where first_name in ('prem', 'shivansh')

Select * from students where first_name like '%a';

Select * from students where first_name like '-----a';

Select * from students where gpa between 9.00 and 9.99;

Select major, count(*) as total_count from student where major = 'computer science';

Select concat(first_name+' '+last_name) as Complete_name from student where gpa between 8.5 and 9.5

Select major, count(major) from student group by major order by count(major) desc;

Select st.name, sh.scholarship_amount from students st inner join scholarship on st.sid = sh.sid;

Select * from student where student_id %2 !=0

Select all from student s left join scholarship on s.sid = sh.sid;

Select * from student order by gpa desc limit 5;

Select * from student order by gpa desc limit 4,1;

Select * from student s1 where 4 = (

Select count(distinct(s2.gpa))

From student s2

Where s2.gpa>= s1.gpa;

) [link01](#) AND [link02](#)

Select s1.* from student s1, student s2 where s1.gpa = s2.gpa and s1.student_id!= s2.student_id;

Select max(gpa) from student where gpa not in(select max(gpa) from student);

Select * from student union all select * from student order by sid;

Select s.sid from student where s.sid not in (select sh.sid from scholarship);

Select * from student limit (select floor(count (*)/2) from student);

Select major, count(major) from student group by major having count(major) < 4;

Select * from student where sid = (select max(sid) from student);

Select * from (select * from student order by sid desc limit 5) as subquery order by sid;

Select distinct gpa from student s1 where 3>=(select count(Distinct gpa) from student s2 where s2.gpa>=s1.gpa) order by s1.gpa desc;

Select major,max(gpa) from students group by major;

Select fn, gpa from student where gpa = (select max(gpa) from student);

Select curdate()

Select now()

Create table ct as select * from student;

Update student set gpa = 7.5 where major = 'cse'

Select major, avg(gpa) as avg from student group by major;

Select * from student s1 where 3<= (

Select count(distinct(s2.gpa))

From student s2

Where s2.gpa>= s1.gpa;

)

Select * from student order by gpa desc limit 3;

Select major, count(sid) as highgpa from student where gpa>7.5 group by major;

Select * from student where gpa = (select gpa from student where sname = 'mahajan');

A window function operates over a set of rows defined by a window (frame) and returns a value for each row.

Window functions perform calculations across a set of table rows that are somehow related to the current row. Unlike aggregate functions (like SUM() or COUNT()) which collapse multiple rows into a single output row using GROUP BY, **window functions return a value for every single row.**

Think of it like this: a GROUP BY function looks at a group of rows and gives you one summary answer for the whole group. A window function looks at a group of rows (the "window") and gives each row in that group a specific answer based on its position or relation to other rows in that window.

The Core: The OVER() Clause

The magic of window functions lies entirely within the OVER() clause. It defines the "window" or the set of rows the function should consider. The clause has three key components, all of which are optional.

FUNCTION_NAME(arguments) OVER ([PARTITION BY ...] [ORDER BY ...] [frame_clause])

Let's break down each part.

PARTITION BY

This clause divides the rows into logical groups, or "partitions." The window function is then applied independently to each partition. If you omit PARTITION BY, the entire result set is treated as a single partition.

- **Interview Analogy:** Think of PARTITION BY as the equivalent of GROUP BY but for a window function. If you want to calculate the average salary *per department*, you would PARTITION BY department. The calculation for the 'Sales' department will not include any data from the 'Engineering' department.

ORDER BY

This clause sorts the rows *within each partition*. The order is crucial for functions that depend on sequence, such as RANK(), LAG(), and running totals.

- **Key Point:** While it looks like a standard ORDER BY clause, its effect is confined within the OVER() clause. It determines the logical order for the window function's calculation, not the final output order of the query (though they often align).

The Frame Clause: Defining the Window's Boundaries

The window frame is a subset of the current partition. It allows you to define a "sliding" window that moves with the current row. The default frame is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, which can sometimes lead to unexpected results, especially with LAST_VALUE().

The syntax is: frame_type BETWEEN lower_bound AND upper_bound

Frame Types: ROWS, RANGE, and GROUPS

This is a critical distinction often asked in interviews.

- 1. **ROWS:** Defines the frame based on a fixed number of physical rows before and after the current row. It's simple and literal.
 - **Example:** ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING means the current row, the row right before it, and the row right after it.
- 2. **RANGE:** Defines the frame based on a logical offset of values from the current row's value in the ORDER BY column. All rows with the same value in the ORDER BY column are considered peers.
 - **Example:** If you ORDER BY sale_date and the frame is RANGE BETWEEN 1 PRECEDING AND CURRENT ROW, and you have 3 sales on '2025-08-17' and 5 sales on '2025-08-18' (the current row), the frame for any '2025-08-18' row will include all 3 rows from the 17th and all 5 rows from the 18th. It looks at the *value*, not the row count.
- 3. **GROUPS (More Advanced):** Defines the frame based on groups of distinct values in the ORDER BY column. It treats all rows with the same value (peers) as a single group.
 - **Example:** With GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW, if you have 3 sales on the 17th and 5 sales on the 18th (current row), the frame includes all rows from the 17th (as one group) and all rows from the 18th (as another group).

Type	Basis	Interview Takeaway
ROWS	Physical Row Position	Literal and predictable. Use for moving averages over a fixed number of records.
RANGE	Value in ORDER BY column	Best for time-series data where you want to include all events within a certain value range (e.g., all sales on the same day).

Type	Basis	Interview Takeaway
GROUPS	Groups of Peers in ORDER BY	Useful when you need to count distinct preceding/following value groups, not just rows. Shows deep knowledge.

Export to Sheets

Frame Boundaries

- UNBOUNDED PRECEDING: The frame starts at the first row of the partition.
- UNBOUNDED FOLLOWING: The frame ends at the last row of the partition.
- CURRENT ROW: The frame starts or ends at the current row.
- n PRECEDING: The frame starts or ends n rows/values/groups before the current row.
- n FOLLOWING: The frame starts or ends n rows/values/groups after the current row.

Categories of Window Functions

1. Aggregate Functions 📊

These are the familiar aggregate functions used with an OVER() clause.

- SUM(), AVG(), COUNT(), MIN(), MAX()
- **Use Case:** Calculating a **running total**.
 - SUM(amount) OVER (PARTITION BY product_id ORDER BY sale_date) gives you the cumulative sales for each product over time.

2. Ranking Functions 🏆

Used to rank rows within a partition. This is a classic interview question.

Function	How it Handles Ties	Example (Scores: 100, 90, 90, 80)
ROW_NUMBER()	Assigns a unique, sequential number. Ignores ties.	1, 2, 3, 4
RANK()	Assigns the same rank to ties, but skips the next rank(s).	1, 2, 2, 4

Function	How it Handles Ties	Example (Scores: 100, 90, 90, 80)
DENSE_RANK()	Assigns the same rank to ties, but does not skip ranks.	1, 2, 2, 3
NTILE(n)	Divides rows into n approximately equal-sized buckets (e.g., quartiles if n=4).	NTILE(2) might give: 1, 1, 2, 2

Export to Sheets

3. Value Functions ↔

Used to access data from a different row relative to the current row.

- LAG(col, offset, default): "Looks back" offset number of rows in the partition and returns the value of col.
- LEAD(col, offset, default): "Looks forward" offset number of rows.
 - **Use Case:** Calculating month-over-month growth. LAG(sales, 1) OVER (PARTITION BY product_id ORDER BY month) gives you the previous month's sales.
- FIRST_VALUE(col): Returns the value of col from the first row in the window frame.
- LAST_VALUE(col): Returns the value of col from the last row in the window frame.
 - **Interview Trap:** Remember the default window frame (RANGE ... CURRENT ROW). To get the true last value of a partition, you often need to specify the frame explicitly: LAST_VALUE(col) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING). Knowing this detail shows expertise.

4. Statistical Functions (Distribution)

These are less common but good to know. They describe the relative position of a row.

- CUME_DIST(): Calculates the cumulative distribution of a value within a group of values. Result is between 0 and 1. It represents the number of rows with values less than or equal to the current row's value, divided by the total number of rows. P(textvaluelex)
 - PERCENT_RANK(): Calculates the relative rank of the current row as a percentage. The formula is $(\text{rank} - 1) / (\text{total_rows} - 1)$. The first row is always 0.
-

Project Use Case: E-Commerce Analytics Dashboard 🚀

Imagine you're building a backend for an e-commerce platform. You have a sales table: (sale_id, product_id, sale_date, amount, region).

Business Requirement: Create an analytical view that, for each sale, shows:

1. A running total of sales for that product.
2. The rank of that sale's amount within its region.
3. The sales amount from the previous day for the same product in the same region (to track daily velocity).
4. The total sales for that region for the entire period, shown on every row.

The SQL Solution:

SQL

WITH ProductSalesAnalysis AS (

SELECT

sale_id,

product_id,

region,

sale_date,

amount,

-- 1. Running total of sales for the product, ordered by date

SUM(amount) OVER (

PARTITION BY product_id

ORDER BY sale_date

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

) AS running_total_product,

-- 2. Rank of the sale amount within its region

DENSE_RANK() OVER (

```

PARTITION BY region

ORDER BY amount DESC

) AS sale_rank_in_region,

-- 3. Previous day's sales for the same product and region
LAG(amount, 1, 0) OVER (

    PARTITION BY product_id, region

    ORDER BY sale_date

) AS prev_day_sales_same_product,

-- 4. Total sales for the entire region, shown on every row
SUM(amount) OVER (

    PARTITION BY region

) AS total_region_sales

FROM

    sales

)

SELECT * FROM ProductSalesAnalysis;

```

Why this is powerful for an interview:

- **Demonstrates Practicality:** You're not just reciting definitions; you're solving a multi-faceted business problem efficiently.
 - **Efficiency:** This single query replaces what would have required multiple complex self-joins, subqueries, or temporary tables, which is far less performant and harder to maintain.
 - **Shows Nuance:** You can explain your choice of DENSE_RANK() (to avoid skipping ranks), the explicit ROWS frame for the running total (for clarity), and the use of multiple columns in the PARTITION BY for LAG().
-

```
mysql> select d.dept_name, max(e.salary) as max_salary from Employees e
```

```
-> JOIN Departments d
```

```
-> on d.dept_id = e.dept_id
```

```
-> group by d.dept_id;
```

dept_name	max_salary
HR	50000
Finance	60000
IT	70000
Sales	65000

```
mysql> select e.*, max(salary) over() as max_salary from Employees e;
```

emp_id	emp_name	dept_id	salary	hire_date	max_salary
101	Alice	1	50000	2020-01-10	70000
102	Bob	2	60000	2019-03-15	70000
103	Charlie	3	70000	2018-07-20	70000
104	David	4	55000	2021-05-25	70000
105	Eva	4	65000	2019-11-30	70000

5 rows in set (0.01 sec)

```
mysql> select e.*, max(salary) over(partition by dept_id) as max_salary from Employees e;
```

partition by: creates one window and apply max(salary) to each of those windows

```
mysql> select e.*, max(salary) over(partition by dept_id) as max_salary from Employees e;
```

emp_id	emp_name	dept_id	salary	hire_date	max_salary
101	Alice	1	50000	2020-01-10	50000
102	Bob	2	60000	2019-03-15	60000
103	Charlie	3	70000	2018-07-20	70000
104	David	4	55000	2021-05-25	65000
105	Eva	4	65000	2019-11-30	65000

ROW_NUMBER()

Assigns unique value to each record

```
mysql> select e.*, row_number()
-> over(partition by dept_id) as rn from Employees e;
```

emp_id	emp_name	dept_id	salary	hire_date	rn
101	Alice	1	50000	2020-01-10	1
102	Bob	2	60000	2019-03-15	1
103	Charlie	3	70000	2018-07-20	1
104	David	4	55000	2021-05-25	1
105	Eva	4	65000	2019-11-30	2

5 rows in set (0.00 sec)

Departments will be formed as groups this row_number() function and assign unique values to each group,

```
select e.*, row_number() over(partition by dept_id) as rn from Employees e;
```

```
select * from
```

```
(      select e.*, row_number()  
        over(partition by dept_name order by emp_id)  
        as rn from Employee e      )
```

```
x where x.rn <3;
```

RANK:-

```
Select e.*,
```

```
Rank() over(partition by dept_name order by salary desc) as rnk
```

```
From Employee e;
```

```
mysql> Select e.*,  
        -> Rank() over(partition by dept_name order by salary desc) as rnk  
        -> From Employee e;
```

emp_ID	emp_NAME	DEPT_NAME	SALARY	rnk
120	Monica	Admin	5000	1
101	Mohan	Admin	4000	2
108	Maryam	Admin	4000	2
113	Gautham	Admin	2000	4

→these are in order based on ORDER BY desc according to that rank will be given if any duplicate found it gives same rank and skip those many ranks as duplicates and resumes from them as shown above

Top 3 employees from each department earn max salary

```
Select * from (
```

```
Select e.*,
```

```
Rank() over(partition by dept_name order by salary desc) as rnk
```

```
From Employee e) x)
```

```
Where x.rnk<4;
```

→ Dense Rank will not skip a value for duplicates

LAG: fetches previous employee salary

```
select e.*,
```

```
-> lag(salary) over (partition by dept_name order by emp_id)
```

```
-> as prev_emp_sal
```

```
-> from employee e;
```

lag(salary,2,0)

2 – fetches two records previous to the current record,

0 – default value if previous two not present

LEAD: same as lag but, it fetches “NEXT” one instead of previous

-- Fetch A query to display if the salary of an employee is higher, lower or equal to the previous Employee

```
Select e.*,
```

```
Lag(salary) over (partition by dept_name order by emp_id) as prev_emp_sal,
```

```
Case
```

```
when e.salary>lag(salary) over (partition by dept_name order by emp_id) then 'higher  
than previous employee'
```

```
when e.salary<lag(salary) over (partition by dept_name order by emp_id) then 'lower  
than previous employee'
```

```
when e.salary=lag(salary) over (partition by dept_name order by emp_id) then 'same as  
previous employee'
```

```
End sal_range
```

```
From employee e;
```

FIRST_VALUE: to fetch first value withing a partition

First_value write query to display most expensive product under each category

Select *,

first_value(product_name) over(partition by product_Category order by price desc) as
most_expensive

From product;

LEAST_VALUE: To fetch least expensive product under each category

Select *,

first_value(product_name) over(partition by product_Category order by price desc) as
most_expensive,

last_value(product_name) over(partition by product_Category order by price desc) as
least_expensive

From product;

This last_value doesn't give same min value for all records of the category

It gives min of till the record under each category

FRAME CLAUSE

Select *,

first_value(product_name) over(partition by product_Category order by price desc) as
most_expensive,

last_value(product_name)

over(

partition by product_Category order by price desc

range between unbounded preceding and current row

)

as least_expensive

From product;

---- default SQL will execute this command

- **unbounded preceding** means first tuple of partition
- within partition it gets the least value from first tuple of partition to current row
- instead of current row use **unbounded following** it takes till last row.
- preceding – all the records prior to current row
- Following – all the records after current row

```

Select *,
first_value(product_name) over(partition by product_Category order by price desc) as
most_expensive,
last_value(product_name)
over(
    partition by product_Category order by price desc
    range between unbounded preceding and unbounded following
)
as least_expensive

```

From product;

- instead of range you can you **rows**
- range: if you mention current row.. a set of records have same value (mentioned in partition of over clause) it considers three records as current row and displays minimum among them
- rows: it strictly considers the current row.

```

Select *,
first_value(product_name) over(partition by product_Category order by price desc) as
most_expensive,
last_value(product_name)
over(
    partition by product_Category order by price desc
    rows between unbounded preceding and unbounded following
)

```

```

)
as least_expensive
From product where product_category = 'Phone';
---
Select *,
first_value(product_name) over(partition by product_Category order by price desc) as
most_expensive,
last_value(product_name)
over(
    partition by product_Category order by price desc
    rows between 2 preceding and 2 following #considers 2 rows prior to the current
row
)
as least_expensive
From product where product_category = 'Phone';

```

```

Select *,
first_value(product_name) over w as most_expensive,
last_value(product_name) over w as least_expensive
From product where product_category = 'Phone'
Window w as (partition by product_Category order by price desc
range between unbounded preceding and unbounded following);

```

Nth value,

Write query to display the second most expensive product under each category

Select *,
 first_value(product_name) over w as most_expensive,
 last_value(product_name) over w as least_expensive,
 nth_value(product_name,2) over w as second_most_expensive
 From product where product_category = 'Phone'
 Window w as (partition by product_Category order by price desc
 range between unbounded preceding and unbounded following);

NTILE:

Group together few records as buckets

Write a query to segregate all the expensive phones, mid range phones and the cheaper phones

Select *, Ntile(3) over(order by price desc) as buckets From product where
 product_category = Phone;

select product_name,
 case when x.buckets = 1 then 'Expensive Phones'
 when x.buckets = 2 then 'mid range Phones'
 when x.buckets = 3 then 'cheaper Phones' END phone_category
 from(
 Select *,
 Ntile(3) over(order by price desc)
 as buckets From product
 where product_category = 'Phone') x;

CUM_DIST: used to identify the distribution percentage of each record with respect to all the rows within the result set.

Provides the value bw range of 0 and 1

--- query to fetch all the products which are constituting the first 30%
--- of the data in products table based on price.

```
select *,  
  
cume_dist() over(order by price desc) as cume_distribution,  
  
round(cume_dist() over(order by price desc)*100,2) as cume_dist_percentage  
  
from product;
```

```
mysql> select *,  
-> cume_dist() over(order by price desc) as cume_distribution,  
-> round(cume_dist() over(order by price desc)*100,2) as cume_dist_percentage  
-> from product;
```

product_category	brand	product_name	price	cume_distribution	cume_dist_percentage
Laptop	Dell	XPS 17	2500	0.037037037037037035	3.7
Laptop	Dell	XPS 15	2300	0.07407407407407407	7.41
Laptop	Microsoft	Surface Laptop 4	2100	0.11111111111111111	11.11
Laptop	Apple	MacBook Pro 13	2000	0.18518518518518517	18.52
Laptop	Dell	XPS 13	2000	0.18518518518518517	18.52
Phone	Samsung	Galaxy Z Fold 3	1800	0.22222222222222222	22.22
Phone	Apple	iPhone 12 Pro Max	1300	0.25925925925925924	25.93
Phone	Samsung	Galaxy Note 20	1200	0.3333333333333333	33.33
Laptop	Apple	MacBook Air	1200	0.3333333333333333	33.33

Top 30% means: like In sales we have situation that

```
SELECT product_name,  
  
CONCAT(cume_dist_percentage, '%') AS cume_dist_percentage  
  
FROM (  
  
SELECT *,  
  
CUME_DIST() OVER (ORDER BY price DESC) AS cume_distribution,  
  
ROUND(CUME_DIST() OVER (ORDER BY price DESC) * 100, 2) AS  
cume_dist_percentage  
  
FROM product  
  
) x  
  
WHERE x.cume_dist_percentage <= 30;
```

Formula = current row no (or Row No with value same as current row)/ total no of rows

PERCENT_RANK: query to identify how much percentage more expensive is “galaxy z fold 3” when compared to all other products

Formula = $\text{current row} - 1 / \text{total no of rows} - 1$

Sub Queries

1. Find the employees who's salary is more than the average salary earned by all employees?

Select * from employee where salary > Select avg(salary) from employee;

Scalar Sub Query:

returns one row and one column

select * from employee e join (select avg(esal) sal from employee) avg_sal on e.esal > avg_sal.sal;

Multiple Row Subquery / Multiple Column :

sub query which returns multiple column and row

Q) Find employees who earn highest salary in each department.

select dept_id, max(esal) from employee group by dept_id;

Select * from employee where (dept_id, esal) in (select dept_id, max(esal) from employee group by dept_id);

Q) find department who don't have any employees

select * from department where id not in (select distinct dept_id from employee);

Co related sub query

a sub query which related to outer query, the processing of your sub query depends the values of outer query.

Q) find employees in each department whose salary is greater than average salary in that department.

select avg(esal) from employee group by dept_id;

- select avg(esal) from employee where dept_name = “abccc”;

```
- select * from employee e1
      where esal > (select avg(esal) from
                    employee e2
                    where e1.dept_id = e2.dept_id);
```

find department who do not have any employees

```
select * from department d where not exists (
select * from employee e where e.dept_id = d.dept_id
);
```

Q) Find Dept who do not have any employees

```
Select * from department d where not exists (select 1 from employee e where e.dept_id
= d.id);
```

Sub Query inside a sub query

Q) find the stores who's sales where better than the average sales across all stores

- 1) find total sales for each store
- 2) find avg sales for all the stores
- 3) compare 1 & 2

```
- Select store_name, sum(price) from sales group by store_name
```

```
Select avg(total_sales) from (select store_name, sum(price) as total_sales) from sales
group by store_name
```

```
Select * from ( select store_name, sum(price) as total_sales From sales group by
store_name) s
```

```
Join (select avg(total_sales) as avg_sales from
(select store_name, sum(price) as total_sales from sales group by store_name) x) a
on s.total_sales > a.sales;
```

**** Use WITH clause when same sub query is repeated ****

With sales as (select store_name, sum(price) as total_sales from sales group by store_name)

Select * from sales join (select avg(total_Sales) as sales from sales x) avg_sales on sales.total_sales > avg_sales.sales;

Subquery we can use in select, from, where , having clauses..

Q) fetch all employee details and add remarks to those employees who earn more than avg pay

* Select

Select * , (case when esal > (select avg(esal) from employee) then 'Higher than average' else null end) as remarks from employee;

** not good practise writing a select clause inside a select clause **;

Select * , (case when esal > avg_sal.sal then 'higher than average' else null end) as Remarks from employee cross join (select avg(esal) sal from employee) avg_sal;

■ Having

Q) find the stores who have sold more units than the average units sold by all stores.

Select store_name, sum(quantity) from sales group by store_name

Having sum(quantity) > (select avg(quantity) from sales);

--- INSERT

Insert into employee_history select e.eid, e.ename, d.name, e.esal from employee e join department d on d.id = e.dept_id

Where not exists (select 1 from employee_history eh where eh.eid = e.eid);

--- update

10% increment all the employees in banglore location based on the maximum salary earned by emp in each dept. only consider employees in employee_history table;

